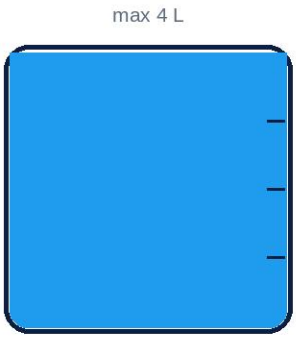
PYNEXIA TECHNOLOGY

INTERNSHIP • HANDS-ON PROJECT • ARTIFICIAL INTELLIGENCE

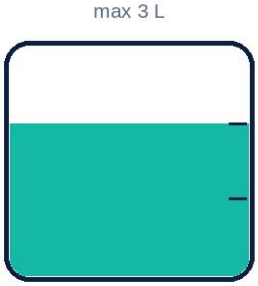
WATER JUG PROBLEM

State Space Search using Breadth-First Search (BFS) and Depth-First Search (DFS) — with a complete, tested Python implementation.



4 L

JUG X • 4 LITRES



2 L

JUG Y • 3 LITRES

TARGET

2 L

measured exactly in either jug

VERIFIED RESULT

BFS solves it in 4 steps → (0,0) → (0,3) → (3,0) → (3,3) → (4,2)

SUBMITTED BY

Sehar Naz

Intern • Pynexia Technology

Project: Hands-On Projects & Assignments (4) • Difficulty: Easy • Language: Python 3

Resource: Water Jug Problem — Search Space Guide (Wikipedia)

Every result in this report was produced by running the code included inside it.

INSIDE THIS PROJECT

Contents & submission snapshot

Everything the assignment asked for — and exactly where to find it in this document.

01	**Project Overview & Problem Statement** What the problem is, and the four tasks set in the brief	3
02	**State Representation & Rules** States, operators, goal test and solvability	4
03	**The Solution at a Glance** The 4-step BFS answer, shown jug by jug	5
04	**Breadth-First Search (BFS)** How BFS explores level by level - shortest path	6
05	**Depth-First Search (DFS)** How DFS dives deep - a valid, longer path	7
06	**Python Implementation** Complete, runnable source code	8
07	**Verified Program Output** Actual terminal output of the submitted code	11
08	**Results & Analysis** BFS vs DFS compared on the same problem	12
09	**Applications, Conclusion & References** Where this is used, and what was learned	13

ASSIGNMENT COVERAGE — ALL FOUR TASKS

✓ **Represent the problem with two fixed-capacity containers**	§02 • Jug X = 4 L, Jug Y = 3 L
✓ **Model each state as the amount of water in both jugs**	§02 • State = (x, y)
✓ **Implement state space search (BFS / DFS) for the target**	§04–§06 • Both algorithms, tested
✓ **Display the exact sequence of state transitions**	§03 & §07 • Step-by-step tables

A classic AI search problem

The Water Jug Problem is one of the foundational problems in Artificial Intelligence. It looks simple — two jugs and an amount of water to measure — but solving it requires exactly the machinery AI uses on far larger problems: representing states, applying rules, and searching systematically for a goal.

THE CHALLENGE IN ONE LINE

You have a 4-litre jug and a 3-litre jug, an unlimited tap and a drain — but no markings on either jug. Measure exactly 2 litres.

WHY THIS PROBLEM MATTERS

- It turns a puzzle into a search problem: a start state, a goal test, and a fixed set of actions — the same structure behind route-finding, planning and puzzle-solving AI.
- Because the jugs are unmarked, you cannot 'just pour 2 litres'. The target amount can only appear as a by-product of filling, emptying and pouring — which is what makes search necessary.
- It has a clean mathematical solvability test (Bézout's identity): a target is reachable only if it is a multiple of gcd(jug capacities). Here gcd(4, 3) = 1, so every whole litre from 0 to 4 is measurable — including our target, 2 L.

PROBLEM STATEMENT — AS SET IN THE BRIEF

1. Represent the water jug problem using two containers with fixed capacities.
2. Model each state as the amount of water in both jugs.
3. Implement state space search (BFS/DFS) to reach the target quantity.
4. Display the exact sequence of state transitions that leads to the solution.

PROJECT SPECIFICATION

Jug X capacity	4 litres
Jug Y capacity	3 litres
Target quantity	2 litres (in either jug)
Initial state	(0, 0) — both jugs empty
Goal state	Any state where x = 2 or y = 2
Algorithms	BFS and DFS (both implemented)
Language / tools	Python 3 — standard library only (collections, math)

Modelling the problem as a state space

A state space turns the physical puzzle into data a computer can search. Every fact that matters about the jugs at any moment fits into a single pair of numbers.

A STATE IS A PAIR

State = (x , y)

x = litres in Jug X (0 to 4) • y = litres in Jug Y (0 to 3)

HOW BIG IS THE SPACE?

At most (4+1) × (3+1) = 20 possible states exist. The search only ever visits states reachable by legal pours — small enough to explore completely, which guarantees the search terminates.

INITIAL STATE, GOAL TEST & ACTIONS

- Initial state — (0, 0). Both jugs start empty. Every solution is a path from here through the state space.
- Goal test — x = 2 or y = 2. The search stops the moment either jug holds exactly the target 2 litres.
- Actions (operators). Six legal moves generate every next state. A pour always stops when the source is empty or the destination is full — never mid-way, because the jugs are unmarked.

THE SIX PRODUCTION RULES

#	Action	Condition	Resulting state
1	Fill Jug X	x < 4	(4, y)
2	Fill Jug Y	y < 3	(x, 3)
3	Empty Jug X	x > 0	(0, y)
4	Empty Jug Y	y > 0	(x, 0)
5	Pour X → Y	x > 0 and y < 3	Pour min(x, 3 − y) litres
6	Pour Y → X	y > 0 and x < 4	Pour min(y, 4 − x) litres

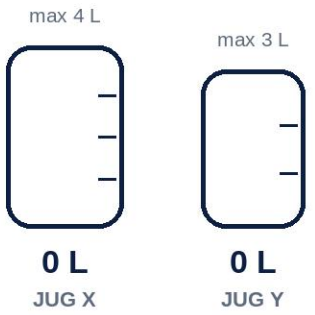
SOLVABILITY — CHECK BEFORE YOU SEARCH (BÉZOUT'S IDENTITY)

A target can be measured only if it is a multiple of gcd(capacities). gcd(4, 3) = 1 and 2 is a multiple of 1 — so this instance is guaranteed solvable before a single state is explored. If the target were, for example, requested from jugs of 4 L and 2 L (gcd = 2), an odd target like 1 L or 3 L would be impossible, and no search could ever find it.

Solved in 4 steps — jug by jug

This is the shortest possible solution, found by Breadth-First Search and verified by running the code on page 8. Each panel shows both jugs after that step.

START



(0 , 0)

Both jugs empty

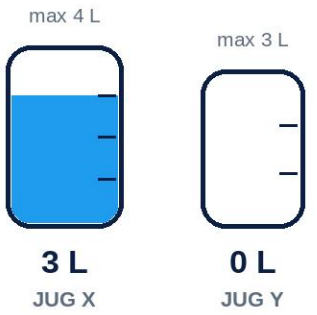
STEP 1 — FILL JUG Y



(0 , 3)

Jug Y filled to its full 3 L

STEP 2 — POUR Y → X



(3 , 0)

All 3 L move into Jug X

STEP 3 — FILL JUG Y



(3 , 3)

Jug Y filled again to 3 L

STEP 4 — POUR Y → X GOAL



(4 , 2)

Jug X takes only 1 L (its free space) — exactly 2 L remain in Jug Y

THE EXACT TRANSITION SEQUENCE (REQUIRED OUTPUT)

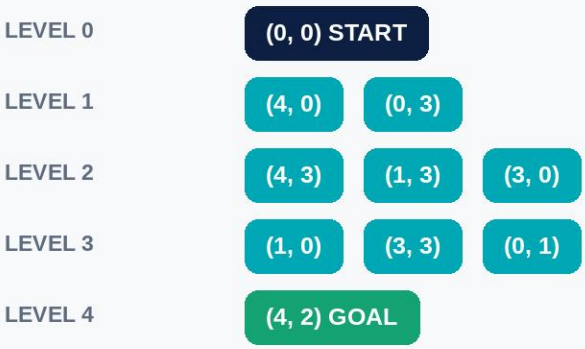
(0, 0) → (0, 3) → (3, 0) → (3, 3) → (4, 2) — 2 L measured in Jug Y

No shorter solution exists: BFS explores states level by level, so the first goal it reaches is always a shortest path.

BFS — level by level, shortest first

BFS keeps its frontier in a queue (first-in, first-out). It expands the start state, then all states one step away, then all states two steps away — so the first time it reaches a goal, that path is guaranteed to be a shortest one.

BFS EXPLORATION BY LEVEL



The goal (4, 2) is generated from (3, 3): pouring Jug Y into Jug X, which has only 1 L of free space, leaves exactly 2 L behind in Jug Y. States are marked visited the moment they are generated, so no state is ever expanded twice.

ORDER IN WHICH BFS EXPANDED STATES

(0,0) → (4,0) → (0,3) → (4,3) → (1,3) → (3,0) → (1,0) → (3,3) → (0,1) → (4,2) — 10 states expanded

PSEUDOCODE

```
function BFS(start):
    queue ← [start]          # FIFO frontier
    visited ← {start};  parent[start] ← none
    while queue is not empty:
        state ← queue.dequeue()
        if state is a goal:  return path via parent
        for each (next, action) in successors(state):
            if next not in visited:
                visited.add(next);  parent[next] ← state
                queue.enqueue(next)
    return failure
```

- Completeness: BFS always finds a solution if one exists, because it systematically covers the whole space level by level.
- Optimality: with equal step costs, the first solution BFS returns uses the fewest possible steps — here, 4.
- Cost: it must remember every visited state, so its memory use grows with the size of the frontier — O(b^d) in the standard notation (branching factor b, solution depth d).

DFS — dive deep first

DFS keeps its frontier in a stack (last-in, first-out). Instead of widening level by level, it commits to one branch and follows it as deep as it goes, backtracking only when a branch is exhausted. Its textbook form uses very little memory — but the first solution it finds need not be the shortest.

THE PATH DFS FOUND — VALID, BUT LONGER (6 STEPS)

STEP	STATE (x, y)	ACTION
0	(0, 0)	Start — both jugs empty
1	(4, 0)	Fill Jug X (4 L)
2	(1, 3)	Pour Jug X → Jug Y (3 L) — Jug Y fills, 1 L stays in X
3	(1, 0)	Empty Jug Y
4	(0, 1)	Pour Jug X → Jug Y (1 L)
5	(4, 1)	Fill Jug X (4 L)
6	(2, 3)	Pour Jug X → Jug Y (2 L) — Jug Y fills, exactly 2 L stay in Jug X

DFS expansion order: (0,0) → (4,0) → (4,3) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3) — only 8 states expanded, because diving straight down one branch reached a goal quickly. The price is path quality: 6 steps instead of BFS's optimal 4.

PSEUDOCODE

```
function DFS(start):
    stack ← [start]          # LIFO frontier
    visited ← {start};  parent[start] ← none
    while stack is not empty:
        state ← stack.pop()
        if state is a goal:  return path via parent
        for each (next, action) in successors(state):
            if next not in visited:
                visited.add(next);  parent[next] ← state
                stack.push(next)
    return failure
```

- Memory: textbook tree-search DFS needs only about O(b x d) for the current path and its siblings. This implementation is graph search, however — it also keeps a visited set and a parent map, so its memory is bounded by the number of distinct states: at most 20 here. That small extra cost is what guarantees the search terminates.
- Not optimal: the first goal reached can sit at the end of a long detour, exactly as happened here (6 steps vs 4).
- Same goal test, same operators, same visited-set discipline — the only difference from BFS is queue vs stack. That single change completely alters the search behaviour, which is the central lesson of this project.

Complete source code — part 1

Program setup, the six production rules as a successors() function, and the goal test. The file is also submitted separately as Water_Jug_Problem.py.

```
"""
Water Jug Problem - State Space Search (BFS & DFS)
Pynexia Technology - Internship Hands-On Project
Author: Sehar Naz

Problem: Jug X = 4 litres, Jug Y = 3 litres, Target = 2 litres
State: (x, y) = amount of water in Jug X and Jug Y
Initial state: (0, 0) | Goal test: x == 2 or y == 2
"""
from collections import deque
import math

CAP_X, CAP_Y, TARGET = 4, 3, 2


def successors(state):
    x, y = state
    moves = []
    # 1. Fill Jug X
    if x < CAP_X:
        moves.append(((CAP_X, y), "Fill Jug X (4L)"))
    # 2. Fill Jug Y
    if y < CAP_Y:
        moves.append(((x, CAP_Y), "Fill Jug Y (3L)"))
    # 3. Empty Jug X
    if x > 0:
        moves.append(((0, y), "Empty Jug X"))
    # 4. Empty Jug Y
    if y > 0:
        moves.append(((x, 0), "Empty Jug Y"))
    # 5. Pour X -> Y
    if x > 0 and y < CAP_Y:
        pour = min(x, CAP_Y - y)
        moves.append(((x - pour, y + pour), f"Pour Jug X -> Jug Y ({pour}L)"))
    # 6. Pour Y -> X
    if y > 0 and x < CAP_X:
        pour = min(y, CAP_X - x)
        moves.append(((x + pour, y - pour), f"Pour Jug Y -> Jug X ({pour}L)"))
    return moves


def is_goal(state):
    return state[0] == TARGET or state[1] == TARGET


```

How to read it: successors(state) is the heart of the program — it applies the six rules from page 4 to one state and returns every legal next state together with a plain-English description of the action, which is later used to print the solution sequence. Every statement sits on its own line, so the logic can be followed top to bottom.

Complete source code — part 2

The BFS and DFS engines — identical in structure except for queue vs stack — and path reconstruction via the parent map.

```
def bfs(start=(0, 0)):
    queue = deque([start])
    visited = {start}
    parent = {start: (None, "Start - both jugs empty")}
    order = []
    while queue:
        state = queue.popleft()
        order.append(state)
        if is_goal(state):
            return reconstruct(parent, state), order, visited
        for nxt, action in successors(state):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = (state, action)
                queue.append(nxt)
    return None, order, visited


def dfs(start=(0, 0)):
    stack = [start]
    visited = {start}
    parent = {start: (None, "Start - both jugs empty")}
    order = []
    while stack:
        state = stack.pop()
        order.append(state)
        if is_goal(state):
            return reconstruct(parent, state), order, visited
        for nxt, action in reversed(successors(state)):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = (state, action)
                stack.append(nxt)
    return None, order, visited


def reconstruct(parent, goal):
    path = []
    cur = goal
    while cur is not None:
        prev, action = parent[cur]
        path.append((cur, action))
        cur = prev
    return list(reversed(path))
```

Note the symmetry: bfs() removes from the left of a queue (popleft), dfs() pops from the end of a stack. Everything else — visited set, parent map, goal test — is shared, which is exactly the point made on page 7.

Complete source code — part 3

The validate() checker, the output printer, and the main program. validate() asserts that a returned path starts at (0, 0), that every single transition is a legal move, and that the final state is a goal — so the word 'tested' in this report is backed by code, not just by eye.

```
def validate(path, name):
    """Assert that a returned path is a genuine solution:
    it starts at (0, 0), every step follows one legal move
    generated by successors(), and the last state is a goal."""
    assert path, f"{name}: empty path"
    assert path[0][0] == (0, 0), f"{name}: path must start at (0, 0)"
    for (state, _), (nxt, action) in zip(path, path[1:]):
        legal = dict(successors(state))
        assert nxt in legal, f"{name}: illegal move {state} -> {nxt}"
        assert legal[nxt] == action, f"{name}: wrong label for {nxt}"
    assert is_goal(path[-1][0]), f"{name}: last state is not a goal"
    return True


def show(name, result):
    path, order, visited = result
    print(f"\n{'=' * 64}\n{name} SOLUTION\n{'=' * 64}")
    print(f"States explored (in order): {order}")
    print(f"Total distinct states visited: {len(visited)}")
    if not path:
        print("No solution found.")
        return
    print(f"Solution length: {len(path) - 1} steps\n")
    for i, (state, action) in enumerate(path):
        print(f"Step {i}: {state}  -  {action}")
    print(f"\nGoal reached: {path[-1][0]}  ->  {TARGET} litres measured successfully.")
    validate(path, name)
    print(f"Validation: PASSED - every transition is a legal move.")


if __name__ == "__main__":
    print("WATER JUG PROBLEM")
    print(f"Jug X capacity: {CAP_X}L | Jug Y capacity: {CAP_Y}L | Target: {TARGET}L")
    g = math.gcd(CAP_X, CAP_Y)
    print(f"Solvability check: gcd({CAP_X},{CAP_Y}) = {g} ;")
    print(f"{TARGET} is a multiple of {g} -> SOLVABLE")
    show("BREADTH-FIRST SEARCH (BFS)", bfs())
    show("DEPTH-FIRST SEARCH (DFS)", dfs())
```

HOW TO RUN

Save the file as Water_Jug_Problem.py, then run: python Water_Jug_Problem.py - no installation needed, only Python 3 (standard library). Output appears exactly as shown on the next page.

Actual output — not a mock-up

This is the real terminal output produced by running the code on pages 8-10, captured unchanged. It is also submitted as program_output.txt.

```
python  Water_Jug_Problem.py
WATER JUG PROBLEM
Jug X capacity: 4L | Jug Y capacity: 3L | Target: 2L
Solvability check: gcd(4,3) = 1 ;
2 is a multiple of 1 -> SOLVABLE

============================================================
BREADTH-FIRST SEARCH (BFS) SOLUTION
============================================================
States explored (in order): [(0, 0), (4, 0), (0, 3), (4, 3), (1, 3), (3, 0), (1, 0), (3, 3), (0, 1), (4, 2)]
Total distinct states visited: 11
Solution length: 4 steps

Step 0: (0, 0)  -  Start - both jugs empty
Step 1: (0, 3)  -  Fill Jug Y (3L)
Step 2: (3, 0)  -  Pour Jug Y -> Jug X (3L)
Step 3: (3, 3)  -  Fill Jug Y (3L)
Step 4: (4, 2)  -  Pour Jug Y -> Jug X (1L)

Goal reached: (4, 2)  ->  2 litres measured successfully.
Validation: PASSED - every transition is a legal move.

============================================================
DEPTH-FIRST SEARCH (DFS) SOLUTION
============================================================
States explored (in order): [(0, 0), (4, 0), (4, 3), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3)]
Total distinct states visited: 9
Solution length: 6 steps

Step 0: (0, 0)  -  Start - both jugs empty
Step 1: (4, 0)  -  Fill Jug X (4L)
Step 2: (1, 3)  -  Pour Jug X -> Jug Y (3L)
Step 3: (1, 0)  -  Empty Jug Y
Step 4: (0, 1)  -  Pour Jug X -> Jug Y (1L)
Step 5: (4, 1)  -  Fill Jug X (4L)
Step 6: (2, 3)  -  Pour Jug X -> Jug Y (2L)

Goal reached: (2, 3)  ->  2 litres measured successfully.
Validation: PASSED - every transition is a legal move.
```

VERIFIED

Both searches terminate at a state containing exactly 2 litres, and the program itself re-checks every transition: validate() asserts each step is a legal move and prints PASSED under each solution.

BFS vs DFS on the same problem

Both algorithms solved the identical instance with the identical rules. Comparing them side by side shows the classic trade-off at the heart of uninformed search.

Criterion	BFS	DFS
Frontier	Queue (FIFO)	Stack (LIFO)
Solution found	(4, 2) — 2 L in Jug Y	(2, 3) — 2 L in Jug X
Path length	4 steps — optimal	6 steps — not optimal
States expanded	10	8
Optimal?	Yes — guaranteed shortest	No — first path found wins
Complete?	Yes	Yes (finite space, visited set)
Memory	Bounded by state count (max 20)	Bounded by state count (max 20)
Time complexity	O(b^d)	O(b^m) (m = max depth)

WHAT THE NUMBERS TEACH

- DFS expanded fewer states here (8 vs 10) because it dove straight down a lucky branch — but 'fewer expansions' did not mean a better answer. Its solution is 50% longer than optimal.
- BFS paid a little more exploration to buy a guarantee: 4 steps is provably the minimum, because no goal exists at levels 0–3 of the search tree.
- On this tiny 20-state space both are instant. The difference matters at scale: BFS's memory grows with the frontier, while DFS can wander deep into unproductive branches — which is why real systems often use hybrids such as Iterative Deepening or A*.
- Correctness was engineered, not hoped for: marking states visited on generation prevents infinite pour-cycles (e.g. filling and emptying the same jug forever), which is the most common beginner error in this problem.
- A note on the memory figures: the O(b^d) / O(b x d) formulas describe tree search. Because this code is graph search with a visited set, its real memory ceiling is the number of distinct states — never more than 20 in this problem, whatever the algorithm.

Where this leads

The same state-space machinery demonstrated on two jugs scales directly to real AI systems.

REAL-WORLD APPLICATIONS OF STATE SPACE SEARCH

- Route & navigation systems — states are locations, actions are roads, the goal is a destination; BFS-style level expansion underlies shortest-path thinking.
- Puzzle & game solving — Rubik's Cube, sliding-tile puzzles and chess endgames are all searched as state spaces.
- Planning & robotics — a robot's configurations are states; search finds an action sequence that reaches a goal configuration without collision.
- Program verification & scheduling — reachable-state exploration checks that a system can never enter a forbidden state.

CONCLUSION

This project represented the Water Jug Problem with two fixed-capacity containers (4 L and 3 L), modelled every situation as a state (x, y), and implemented both Breadth-First and Depth-First state space search in Python to measure a target of exactly 2 litres. BFS found the optimal 4-step solution — (0,0) → (0,3) → (3,0) → (3,3) → (4,2) — while DFS found a valid 6-step solution, giving a concrete, measured demonstration of the optimality-versus-memory trade-off between the two classic uninformed search strategies. All four tasks in the assignment brief were completed, and every result was verified by executing the submitted code.

REFERENCES

1. Water Pouring Puzzle — Wikipedia (resource provided with the assignment): en.wikipedia.org/wiki/Water_pouring_puzzle — rules, the standard example, reversibility of actions, and Bézout's identity solvability condition.

2. Russell, S. & Norvig, P. — Artificial Intelligence: A Modern Approach — uninformed search strategies (BFS / DFS), state-space formulation and complexity analysis.

3. Cowley, E. B. (1926). 'Note on a Linear Diophantine Equation', American Mathematical Monthly, 33(7) — historical source for the classic jug puzzle, via Wikipedia.

SUBMITTED BY

Sehar Naz • Intern, Pynexia Technology

Deliverables: this project report (PDF) • Water_Jug_Problem.py (runnable source code) • program_output.txt (verified output). All results in this report were generated by running the submitted code — nothing is hand-typed or assumed.